

Investigating and Implementing LibreRouter and LibreMesh: Virtualized and Raspberry Pi-Based Mesh Networking

Your Name
EC3730 Final Project

Abstract—This project investigates LibreRouter and LibreMesh, open-source community networking technologies built on top of OpenWrt. The project explores the architecture, implementation, vulnerabilities, and deployment challenges associated with decentralized mesh networking systems. A virtualized OpenWrt environment was first developed using VMware Workstation Pro to simulate a four-node LibreMesh network topology. The project will later extend the deployment to Raspberry Pi hardware platforms for physical mesh networking experiments and performance evaluation.

I. INTRODUCTION

Community mesh networks provide decentralized communication infrastructure capable of operating independently of centralized Internet Service Providers (ISPs). LibreRouter and LibreMesh were developed to support resilient, community-owned wireless infrastructure using open-source software and commodity hardware.

This project investigates the deployment and operation of LibreMesh-based networking systems using both virtualized and physical implementations. The primary goals include evaluating routing behavior, deployment complexity, security considerations, network resilience, and practical limitations of community mesh networking technologies.

In addition to traditional research analysis, this report maintains a reproducible engineering notebook style documenting implementation steps, configuration procedures, troubleshooting observations, and experimental outcomes throughout the project lifecycle.

II. BACKGROUND RESEARCH

A. OpenWrt

OpenWrt is an open-source Linux-based operating system designed primarily for embedded networking devices such as routers and access points. Unlike traditional vendor firmware, OpenWrt provides a writable filesystem, package management system, and modular architecture allowing users to customize routing, firewall, wireless, and network management functionality.

The OpenWrt ecosystem supports x86 virtualized environments in addition to physical embedded hardware, making it suitable for experimental mesh networking research.

B. LibreMesh

LibreMesh is a framework built on top of OpenWrt that simplifies the deployment of decentralized mesh networking infrastructure. The platform automates many aspects of wireless mesh configuration while supporting routing protocols such as BATMAN-adv and BMX7.

LibreMesh was designed to support community-operated networks where nodes dynamically discover neighboring devices and establish resilient communication paths without centralized infrastructure.

C. LibreRouter

LibreRouter is an open-hardware networking platform developed specifically for community mesh deployments. The platform combines OpenWrt-based firmware with wireless hardware optimized for long-range and community-managed networking applications.

III. LITERATURE REVIEW AND SECURITY ANALYSIS

A. Identified Weaknesses and Vulnerabilities

Several security concerns and operational limitations were identified during preliminary literature review:

- Firmware update vulnerabilities in OpenWrt package management systems
- Mesh routing attacks including route poisoning and malicious node participation
- Wireless sniffing and unauthorized network access
- Resource exhaustion and denial-of-service attacks
- Complexity of maintaining decentralized routing infrastructure

IV. VIRTUALIZED OPENWRT DEPLOYMENT

A. Virtualization Environment

VMware Workstation Pro 17 was selected as the virtualization platform for the initial implementation phase. Virtualization allowed rapid deployment, rollback capability through snapshots, reproducible experimentation, and simplified troubleshooting prior to deployment on physical Raspberry Pi hardware.

B. OpenWrt Image Preparation

The OpenWrt x86_64 ext4 combined image was downloaded from the official OpenWrt release repository. Since VMware does not natively boot raw IMG files efficiently, the OpenWrt disk image was converted from IMG format into VMware VMDK format prior to deployment.

C. Virtual Machine Configuration

A custom virtual machine configuration was selected in VMware Workstation Pro to allow manual control over networking interfaces and storage configuration.

The following VM specifications were used:

TABLE I
INITIAL OPENWRT VM CONFIGURATION

Setting	Value
Guest OS	Other Linux 5.x 64-bit
vCPUs	1
Memory	256 MB
Primary Disk	OpenWrt VMDK
Network Adapter 1	NAT
Network Adapter 2	Host-only

D. Network Interface Design

Two virtual network adapters were configured for each virtual router node:

- NAT adapter for WAN connectivity and package installation
- Host-only adapter for internal mesh backbone communication

This architecture allows the virtual environment to simulate isolated mesh connectivity between router nodes while still maintaining Internet access for software installation and updates.

E. Initial Boot Process

The OpenWrt virtual machine booted successfully after the VMDK disk image was attached to the VMware virtual SCSI controller. During startup, OpenWrt successfully initialized both virtual Ethernet interfaces:

```
eth0 NIC Link is Up
eth1 NIC Link is Up
br-lan entered forwarding state
```

The successful initialization of both interfaces confirmed that VMware virtual networking devices were correctly recognized by the OpenWrt kernel.

F. Implementation Notes

- VMware generated a warning regarding a missing host CD/DVD device during startup.
- The warning was safely ignored because OpenWrt boots directly from the virtual disk image and does not require installation media after deployment.
- OpenWrt successfully booted into the root shell environment.

- The system displayed a migration notice indicating OpenWrt's transition from the legacy `opkg` package manager to the newer `apk` package management system.

V. VIRTUALIZED OPENWRT DEPLOYMENT AND MESH BACKBONE CONSTRUCTION

A. VMware Environment Setup

OpenWrt 25.12.4 x86_64 was deployed inside VMware Workstation Pro 17 using an existing virtual disk image. The initial virtual machine configuration is shown in Table ??.

TABLE II
INITIAL VMWARE OPENWRT CONFIGURATION

Component	Configuration
Guest OS	Other Linux 5.x 64-bit
Memory	256 MB
vCPU	1
Disk	Existing OpenWrt image
Adapter 1	NAT (VMnet8)
Adapter 2	Host-only (VMnet1)

The OpenWrt virtual machine successfully booted and detected two Intel e1000 virtual network interfaces:

- `eth0`
- `eth1`

B. Initial Network Troubleshooting

Initial deployment attempts failed to provide Internet connectivity. Observed symptoms included:

- missing default routes
- failed DNS resolution
- inability to ping external IP addresses
- DHCP inconsistencies

Example failure:

```
ping -c 4 8.8.8.8
Network unreachable
```

Investigation revealed that VMware host-only networking was unintentionally acting as the WAN DHCP source. Multiple adapter configurations caused interface ambiguity and routing instability.

Observed VMware virtual networks included:

- VMnet1: 192.168.61.0/24
- VMnet8: 192.168.111.0/24

At various stages:

- `eth1` received DHCP assignments from the host-only network
- NAT routing was absent
- default routes were not installed

C. Controlled Simplification Strategy

To isolate the networking issue, the second adapter was temporarily removed and the deployment was simplified to a single NAT interface.

After simplification, the OpenWrt node successfully received:

- DHCP assignment
- default gateway
- external DNS access

Working configuration:

```
default via 192.168.111.2 dev eth0
```

Successful validation tests included:

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

All tests completed successfully with zero packet loss.

D. Package Management Failure Analysis

Package management initially failed during APK operations.

Observed errors included:

```
wget exited with error 1
unexpected end of file
```

Further investigation revealed that BusyBox wget lacked HTTPS support:

```
wget https://downloads.openwrt.org/
HTTPS support not compiled in.
```

This explained repeated failures retrieving APK repository metadata over HTTPS.

The issue represented an operational weakness because administrators were effectively forced to downgrade repository transport from HTTPS to HTTP in order to restore package functionality.

Potential security implications include:

- man-in-the-middle attacks
- package tampering
- repository spoofing
- supply-chain compromise

E. Temporary Package Management Recovery

HTTPS functionality was partially restored by manually transferring packages into the VM and installing them locally:

```
apk add --allow-untrusted ./*.apk
```

However, repository index downloads continued failing over HTTPS. Repository transport was therefore temporarily changed from:

```
https://
```

to:

```
http://
```

After modification:

```
apk update
```

successfully completed.

Core administrative packages were then installed successfully:

```
apk add nano
apk add vim
apk add curl
apk add tcpdump
apk add htop
apk add bash
apk add luci
```

F. LuCI Web Interface Deployment

After installation of LuCI, the OpenWrt web interface was initially inaccessible through:

```
http://192.168.1.1
```

Diagnostic analysis using:

```
netstat -tlnp
```

confirmed that the web server was actively listening on ports 80 and 443:

```
0.0.0.0:80 LISTEN
0.0.0.0:443 LISTEN
```

Further analysis revealed that the Windows host system could only directly access the VMware NAT network interface rather than the internal OpenWrt LAN bridge.

The LuCI interface became accessible using:

```
http://192.168.111.130
```

rather than the internal bridge address.

This demonstrated the distinction between:

- internal OpenWrt LAN addressing
- externally reachable VMware NAT addressing
- hypervisor network segmentation behavior

The LuCI interface successfully loaded using:

- username: root
- blank default password

A secure administrative password was configured immediately afterward.

G. Dual-Interface Mesh Backbone Construction

After establishing a stable baseline, a second virtual adapter was carefully reintroduced using VMware host-only networking.

The second interface was assigned as a dedicated mesh backbone network:

```
uci set network.mesh=interface
uci set network.mesh.proto='static'
uci set network.mesh.device='eth1'
uci set network.mesh.ipaddr='10.10.10.1'
uci set network.mesh.netmask='255.255.255.0'
uci commit network
/etc/init.d/network restart
```

Resulting topology:

Routing verification confirmed correct segmentation:

```
default via 192.168.111.2 dev eth0
```

TABLE III
FINAL DUAL-INTERFACE OPENWRT TOPOLOGY

Interface	Address	Purpose
eth0	192.168.111.130/24	VMware NAT / WAN
eth1	10.10.10.1/24	Internal mesh backbone

10.10.10.0/24 dev eth1

This configuration established a stable foundation for future multi-node mesh experimentation.

H. VM Cloning Strategy

To support multi-node LibreMesh experimentation, the stable OpenWrt node was prepared for VMware cloning.

Instead of reinstalling OpenWrt repeatedly, a single stable “golden image” was constructed and prepared for duplication.

Planned virtual nodes include:

- OpenWrt-Node1
- OpenWrt-Node2
- OpenWrt-Node3
- OpenWrt-Node4

Each node will later receive:

- unique hostname
- unique mesh IP address
- unique routing configuration

The shared host-only network will serve as the virtual mesh backbone interconnecting all nodes.

VI. PLANNED LIBREMESH DEPLOYMENT

The next implementation phase will involve:

- Installing LibreMesh packages
- Configuring BATMAN-adv mesh routing
- Cloning the initial VM into four router nodes
- Constructing a multi-hop virtual mesh topology
- Performing connectivity and routing validation tests

VII. FUTURE RASPBERRY PI DEPLOYMENT

The final implementation stage will migrate the virtualized mesh environment onto Raspberry Pi hardware devices running OpenWrt and LibreMesh.

The Raspberry Pi deployment will evaluate:

- Physical wireless mesh formation
- Real-world throughput and latency
- Hardware limitations
- Deployment obstacles
- Inter-group node interoperability

VIII. CONCLUSION

The initial virtualization phase successfully established a functional OpenWrt router environment capable of supporting future LibreMesh experimentation. The implementation demonstrated the practicality of virtualized router deployment using VMware Workstation Pro and established the networking foundation necessary for future multi-node mesh routing experiments.

REFERENCES

- [1] OpenWrt Project, “OpenWrt Documentation,” [Online]. Available: <https://openwrt.org>
- [2] LibreMesh Project, “LibreMesh Documentation,” [Online]. Available: <https://libremesh.org>
- [3] BMX7 Routing Protocol, “BMX7 GitHub Repository,” [Online]. Available: <https://github.com/bmx-routing/bmx7>